

FINAL YEAR PROJECT · FINAL REPORT

AI Voice Appointment System

A multilingual conversational assistant that books, changes and cancels doctor appointments, with a doctor dashboard and a clinic administration panel

Report date	25 September 2026
Languages served	English, Urdu, Roman Urdu and mixed speech
Repository	github.com/HamzaTahir554/AI-voice-appointment-system
Scope of this report	What is implemented in the repository at the date above. Components that were planned but are not implemented are stated as such.

Contents

1	Introduction	12	Speech-to-Text (STT)
2	Objectives	13	Text-to-Speech (TTS)
3	System Overview	14	Asterisk / SIP Telephony
4	System Architecture	15	Testing
5	Intent Detection (mBERT)	16	Security
6	Dialogue Manager	17	Technologies Used
7	LLM Judge and Response Generator	18	Performance Measurements
8	Appointment Backend	19	Limitations and Future Work
9	Firebase / Database	20	Conclusion
10	Doctor Dashboard	A	Appendix A – HTTP API Reference
11	SuperAdmin Dashboard	B	Appendix B – Configuration Variables

Implementation status at a glance

Component	Status	Where
Fine-tuned mBERT intent detection (31 intents)	Implemented	intent_detection/
Dialogue Manager	Implemented	dialog_manager/
LLM judge, response generator, hallucination guard (Ollama)	Implemented	ollama_judge/
Appointment Backend and HTTP API	Implemented	appointment_backend/ , api/
Firebase Firestore persistence	Implemented	firebase/
Doctor Dashboard	Implemented	Dashboard/
SuperAdmin Dashboard (single clinic)	Implemented	Dashboard/ , api/admin.py
Appointment statistics	Implemented	appointment_backend/statistics.py
Dashboard authentication and roles	Prototype	api/auth.py
Patient notifications	Queued, not delivered	firebase/notification_service.py
Speech-to-Text	Not implemented	—
Text-to-Speech	Not implemented	—
Asterisk / SIP telephony	Not implemented	—

1. Introduction

1.1 Project title

AI Voice Appointment System – a multilingual conversational assistant for booking doctor appointments at a clinic in Pakistan.

1.2 Background

Most clinics in Pakistan take appointments by telephone. A receptionist answers, checks the doctor's diary by hand or in a spreadsheet, and writes the booking down. Patients speak Urdu, English, or – very often – Roman Urdu mixed with English words ("Mujhe Dr Ahmed se kal 4 baje appointment chahiye"). Staff time is limited, calls are missed outside opening hours, and double bookings and lost cancellations are common.

1.3 Problem statement

Appointment handling needs to understand what a patient wants in the languages they actually use, apply the clinic's scheduling rules consistently, and keep doctors and clinic staff informed – without a person having to transcribe every request.

1.4 Proposed solution

A conversational pipeline that takes what the patient says, classifies their intent with a fine-tuned multilingual BERT model, runs a stateful dialogue to collect the doctor, date and time, applies the clinic's rules in a dedicated appointment backend, stores everything in Firebase Firestore, and phrases the reply with a local large language model that is prevented from changing any fact. Doctors manage their own day through a web dashboard; a clinic administrator manages the doctors, the clinic and every appointment through a separate administration area.

1.5 Motivation

- Serve patients in their own language, including Roman Urdu and code-mixed speech that off-the-shelf English systems handle poorly.
- Keep the rules deterministic: a language model is good at wording and bad at being trusted with a diary, so the booking logic is ordinary tested code.
- Give doctors and the clinic an accurate, shared view of the diary, whether an appointment was made through the assistant or on the dashboard.

Scope note. The design targets telephone calls (Asterisk/SIP, speech recognition and speech synthesis). Those three stages are **not implemented** in this repository. The implemented pipeline takes the patient's words as *text* and returns the reply as *text*; Sections 12–14 describe the exact state of each.

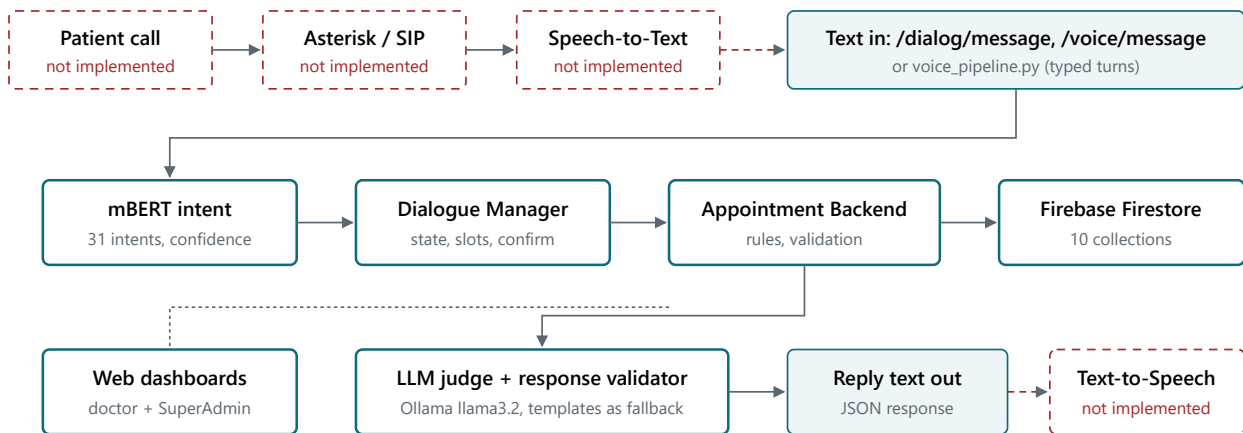
2. Objectives

The objectives below are the ones the implemented system meets.

1. **Understand multilingual requests.** Classify a patient's utterance into one of 31 intents in English, Urdu (Arabic script), Roman Urdu or a mixture, using a fine-tuned mBERT model.
2. **Hold a real conversation.** Track what has been said across turns, ask only for missing information, confirm before acting, and recover from corrections and unclear input.
3. **Enforce clinic rules in one place.** Book, cancel and reschedule only through a backend that validates dates, working hours, doctor availability and double bookings.
4. **Persist reliably.** Store patients, doctors, the clinic, schedules, appointments, conversations and notifications in Firebase Firestore, with an in-memory stand-in for offline development and tests.
5. **Word replies naturally but safely.** Use a local LLM through Ollama to phrase replies, and reject any wording that contradicts the backend's result.
6. **Serve the doctors.** Give each doctor a dashboard for their own appointments, patients, working hours, leave, profile, notifications and statistics.
7. **Serve the clinic.** Give a single administrator control of the one clinic, every doctor, their sign-in credentials, their leave, every appointment and clinic-wide statistics.
8. **Protect access.** Separate the two roles on the server, keep credentials out of source code and the browser, and hash stored passwords.

3. System Overview

The intended end-to-end flow, and what exists today:



Solid border: implemented. Dashed red border: designed but not implemented.
The dashboards reach the backend through the same FastAPI service (/dashboard/*, /admin/*).

3.1 One conversation turn, as implemented

1. The patient's words arrive as text at `POST /dialog/message` (or `/voice/message`, or the interactive `voice_pipeline.py`).
2. The **intent router** runs the fine-tuned mBERT classifier and returns an intent and a confidence.
3. The **Dialogue Manager** updates the session state, extracts entities (doctor, date, time, appointment ID, patient details), and decides the next action: ask for a missing slot, check availability, ask for confirmation, or act.
4. Only after confirmation does it call the **Appointment Backend**, which validates the request and reads or writes **Firestore**.
5. The backend's result goes to the **Ollama judge**, which phrases a reply in the caller's language. The **response validator** rejects any wording whose times, dates or IDs differ from the result, in which case a deterministic template is used instead.
6. The reply is returned as text, together with the intent, confidence, action and slot state.

Every appointment written this way appears in the doctor's dashboard and the administrator's views, because both read the same Firestore collections through the same backend.

4. System Architecture

The system is one Python code base served by a single FastAPI process, plus a static web front end served by that process at `/ui`. There is one backend: the voice pipeline and both dashboards call the same Appointment Backend and the same Firestore data layer.

Layer	Package	Responsibility
Configuration	<code>config.py</code>	Every tunable value, read from environment variables with safe defaults; statuses, actions and collection names.
Intent detection	<code>intent_detection/</code>	Data preparation, training, evaluation and inference for the mBERT classifier.
Dialogue	<code>dialog_manager/</code>	Session state, slot filling, entity validation, semantic fallback for low confidence, emergency escalation.
Wording	<code>ollama_judge/</code>	Ollama client, prompts, language handling, spoken templates, response validator.
Business rules	<code>appointment_backend/</code>	Booking, cancellation, rescheduling, availability, validation, doctor leave cascade, statistics.
Persistence	<code>firebase/</code>	Repository abstraction over Firestore (and an in-memory equivalent), one service per collection.
HTTP service	<code>api/</code>	FastAPI application, dialogue and voice endpoints, dashboard and administration APIs, authentication.
Front end	<code>Dashboard/</code>	Plain HTML, CSS and JavaScript dashboards for doctors and the administrator.
Entry points	<code>voice_pipeline.py</code> , <code>scripts/</code>	Interactive console pipeline, demo, evaluation, performance and Firebase checks, test runner, report builder.

4.1 How the components communicate

- **In-process calls.** Within the API process, the Dialogue Manager calls the intent router, the Appointment Backend and the Ollama judge as ordinary Python objects.
- **HTTP to Ollama.** The judge calls the local Ollama server's REST API with `httpx` (default `http://localhost:11434`).
- **Firestore Admin SDK.** The `firebase/` repository talks to Firestore with the server-side Admin SDK. The browser never talks to Firestore directly.
- **HTTP from the browser.** The dashboards call `/auth/*`, `/dashboard/*` and `/admin/*` with a bearer token; every call is scoped on the server to the signed-in role.
- **One result shape.** Every backend operation returns an `OperationResult` (`success`, `operation`, `appointment_id`, `status`, `data`, `error`), which the API maps to HTTP status codes (404, 403, 400, 409, 503).

4.2 Why the language model is last

By the time the LLM runs, the appointment has already been booked, refused or looked up by deterministic code. The LLM can only change the wording; the validator checks the wording against the result. The worst case when the model misbehaves is a plainer sentence, never a wrong booking.

5. Intent Detection (mBERT)

5.1 Model and architecture

Property	Value
Base model	<code>bert-base-multilingual-cased</code> (mBERT)
Head	<code>BertForSequenceClassification</code> – a linear classifier over the pooled [CLS] representation
Encoder	12 transformer layers, hidden size 768
Vocabulary	119,547 WordPiece tokens (shared across 104 languages, including Urdu script)
Output	31 intents; softmax probability used as confidence
Saved model size	679 MB (<code>model.safetensors</code>)

mBERT was chosen because one model covers English, Urdu in Arabic script and Roman Urdu. The training loop is hand-written PyTorch rather than the Transformers `Trainer`, which keeps it independent of API changes between library versions.

5.2 Dataset

Four source files in `Data/` are merged by `intent_detection/src/preprocess.py`, which detects each file's text and label columns, maps their different label vocabularies onto one taxonomy, removes duplicates and assigns each row a script (English, Urdu, Roman Urdu).

Source file	Rows in file	Rows used	Notes
<code>dataset.csv</code>	405	810	Carries a parallel Urdu column, so each row gives two examples
<code>dataset2.csv</code>	2,159	159	Most rows collapse to duplicates after normalisation
<code>dataset3.csv</code>	1,500	118	As above
<code>synthetic_intents.csv</code>	2,377	2,374	Template-generated phrasings for under-represented intents, contrast pairs, natural and Roman-Urdu spelling variants
Total		3,461	

Split	Rows	Script	Rows
Train	2,760	Roman Urdu	1,242
Validation	347	English	1,231
Test	354	Urdu (Arabic script)	987
		Other	1

The split is **group-aware**: an Urdu row and the Roman-Urdu row it was translated from always land in the same split, so the test score cannot be inflated by a near-copy seen in training. The corpus is imbalanced – `book_appointment` has 1,417 rows (41%) and the smallest intent, `provide_patient_gender`, has 18 – which is handled with class weights (5.4).

5.3 Intent classes (31)

Group	Intents
Appointments	<code>book_appointment</code> , <code>cancel_appointment</code> , <code>reschedule_appointment</code> , <code>appointment_status</code> , <code>appointment_confirmation</code> , <code>check_availability</code>
Changes during a booking	<code>change_date</code> , <code>change_time</code> , <code>change_doctor</code>
Doctor information	<code>find_doctor</code> , <code>doctor_information</code> , <code>doctor_specialization</code> , <code>doctor_qualifications</code> , <code>doctor_fee</code> , <code>doctor_unavailable</code>
Clinic information	<code>clinic_location</code> , <code>clinic_timing</code> , <code>clinic_closed</code>
Patient details	<code>provide_patient_name</code> , <code>provide_patient_phone</code> , <code>provide_patient_age</code> , <code>provide_patient_gender</code>
Conversation	<code>greeting</code> , <code>goodbye</code> , <code>thank_you</code> , <code>confirm</code> , <code>deny</code> , <code>help</code> , <code>repeat_information</code> , <code>unclear_request</code>
Safety	<code>emergency</code>

5.4 Training process and hyperparameters

Setting	Value	Reason
Maximum sequence length	64 tokens	The longest utterance is 29 tokens; 128 would waste computation
Batch size	16, with 2 gradient-accumulation steps (effective 32)	Fits a 6 GB GPU
Optimiser	AdamW, learning rate 2×10^{-5} , weight decay 0.01	Standard for BERT fine-tuning
Schedule	Linear, 10% warm-up	
Epochs	20, early stopping on validation macro-F1, patience 5	A small validation set makes macro-F1 noisy between epochs
Loss	Cross-entropy with class weights, inverse frequency to the power 0.5 (a $9 \times$ spread)	Full inverse-frequency weights over-predicted rare intents
Precision	Mixed (fp16)	
Seed	42	Reproducible splits and initialisation

Hardware	NVIDIA GeForce GTX 1660 SUPER, 6 GB; about 233 s per epoch	
Selected checkpoint	Epoch 19 of 20, validation macro-F1 0.8659	Best validation macro-F1

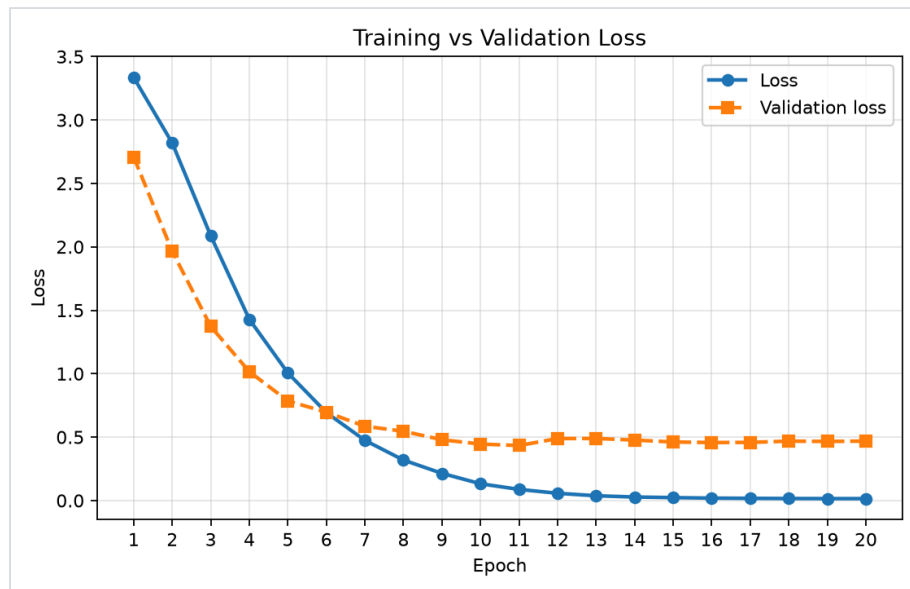


Figure 1 – Training and validation loss per epoch (`intent_detection/outputs/loss_comparison.png`).

5.5 Evaluation on the held-out test set

Measured on the 354-row test set by `intent_detection/src/evaluate.py`
(`intent_detection/outputs/test_metrics.json`):

Metric	Macro average	Weighted average
Accuracy	0.8983	
Precision	0.8680	0.9097
Recall	0.8404	0.8983
F1-score	0.8362	0.8937

Macro averages weight every intent equally, so they are lower than the weighted ones: several intents have only 4–8 test examples, where one mistake moves recall by 12–25 points.

Strongest and weakest intents (test set F1)

Intent	Precision	Recall	F1	Support
book_appointment	0.948	0.993	0.970	148
change_doctor	1.000	1.000	1.000	13
reschedule_appointment	0.889	0.889	0.889	18

check_availability	1.000	0.833	0.909	12
emergency	1.000	0.546	0.706	11
change_time	1.000	0.500	0.667	6
repeat_information	0.500	0.667	0.571	6
goodbye	1.000	0.333	0.500	6
clinic_timing	0.400	0.400	0.400	5

Seven intents reach F1 1.00 (change_date, change_doctor, doctor_unavailable, greeting, provide_patient_age, provide_patient_gender, provide_patient_phone). The weakest are short information questions that differ by a single content word (*timing* versus *location* versus *fee*), and conversational closers with very few examples. **Emergency recall is 0.55**, which is why emergencies are not left to the classifier alone (5.6).

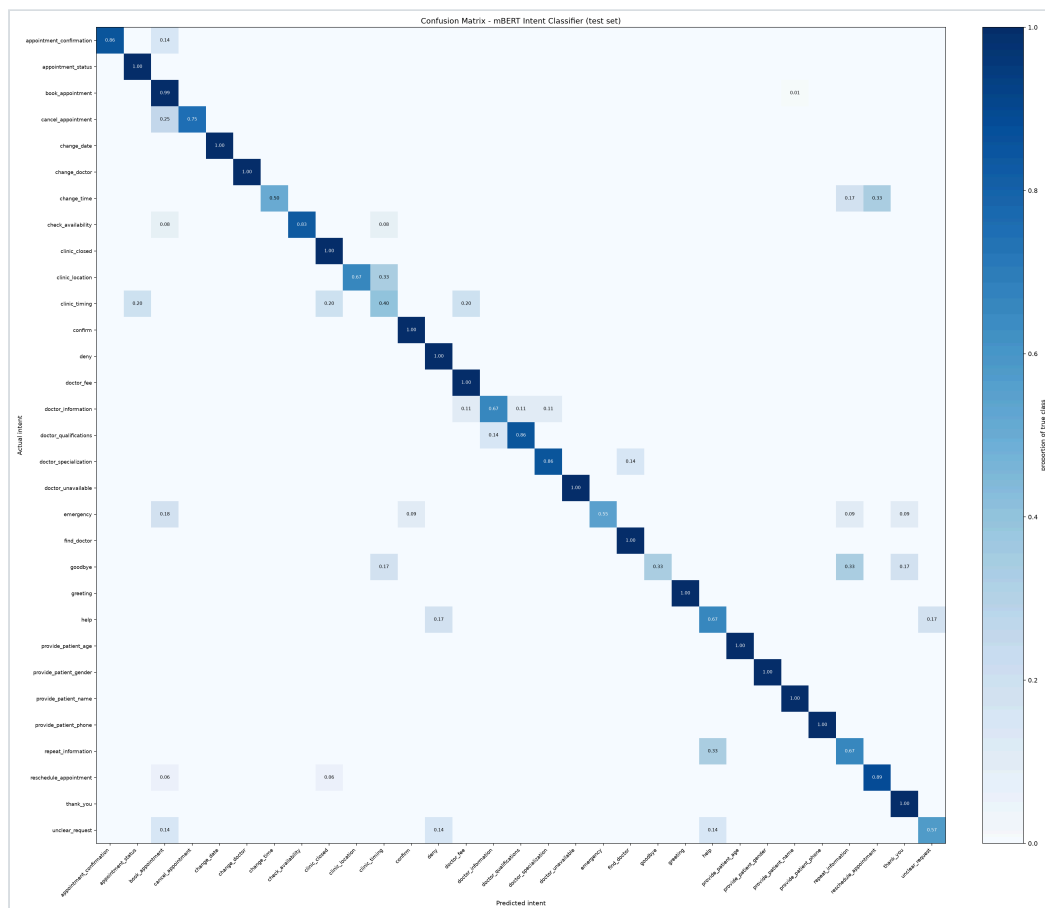


Figure 2 – Normalised confusion matrix on the test set
(intent_detection/outputs/confusion_matrix_normalized.png).

5.6 Confidence handling

Confidence	Behaviour	Setting
≥ 0.80	The intent is acted on.	HIGH_CONFIDENCE

0.60 – 0.79	The intent is used, but a <i>semantic fallback</i> must corroborate it (a recognisable subject and action, and no competing-intent signal); otherwise the assistant confirms what it understood.	CONFIDENCE_THRESHOLD
< 0.60	Treated as unclear; the assistant asks the caller to rephrase instead of touching the database.	
Mid-booking switch	An in-progress booking is only abandoned for a different intent above 0.75, so a stray word does not derail it.	INTENT_SWITCH_THRESHOLD

Emergency safety net. Because the model misses some emergencies phrased in ways it has not seen, the Dialogue Manager also escalates acute-danger phrases (for example chest pain, heavy bleeding, breathlessness) regardless of the predicted intent, with stand-down rules for routine mentions such as "test" or "checkup". It is on by default (`EMERGENCY_SAFETY_NET`).

6. Dialogue Manager

The Dialogue Manager (`dialog_manager/`) turns a sequence of intents into a finished task. It is deterministic code, not a language model.

Module	Responsibility
<code>dialog_manager.py</code>	The conversation controller: routes each turn, chooses the next action, calls the backend.
<code>intent_router.py</code>	Loads the mBERT model and applies the confidence bands.
<code>state_manager.py</code>	Per-session state, persisted in <code>conversation_sessions</code> ; sessions expire after 15 minutes of silence.
<code>slot_manager.py</code>	Which slots each workflow needs (doctor, date, time, appointment ID, patient details) and which are still missing.
<code>validators.py</code>	Entity extraction and validation: doctor names and aliases, relative and absolute dates, spoken times, phone numbers, appointment IDs.
<code>semantic_fallback.py</code>	Corroborates low- and medium-confidence intents; also hosts the emergency safety net.
<code>responses.py</code>	Turns a decision into words, in the caller's language.

6.1 Conversation management

Each turn is processed against the session's state: the active workflow, the filled slots, any pending confirmation and the language the caller is using. Every message is stored in `conversation_messages` (phone numbers masked), and `GET /dialog/transcript/{session_id}` returns a call's transcript.

6.2 Intent handling

The router's intent selects a workflow (book, cancel, reschedule, check status, availability, doctor or clinic information, patient details, conversation). Side questions asked in the middle of a booking ("what is Dr Ahmed's fee?") are answered and the booking then resumes. Correction intents (`change_date` , `change_time` , `change_doctor`) are absorbed into the current booking instead of starting a new one.

6.3 Slot and entity handling

- **Doctor:** matched against the name and aliases stored on each doctor record, longest alias first; an ambiguous name ("Dr Hamza" when two doctors share it) is asked about rather than guessed. A replacement ("Dr Ahmed nahi, Dr Asim") replaces the earlier choice.
- **Date:** relative dates (kal, parson, today, tomorrow), weekday names, and day-of-month forms, resolved in the `Asia/Karachi` time zone.
- **Time:** spoken forms such as "shaam 5 baje" or "4:30 PM".
- **Appointment ID and patient details:** extracted when a workflow needs them.

6.4 Confirmation, clarification and the booking flow

```
greeting -> book_appointment
  -> ASK_FOR_DOCTOR -> ASK_FOR_DATE -> ASK_FOR_TIME    (only the missing slots)
  -> CHECK_AVAILABILITY (backend dry run)
    free:      ASK_CONFIRMATION -> "haan" -> CREATE_APPOINTMENT -> reply with ID
    not free: OFFER_ALTERNATIVES (nearest free slots)
  -> deny at confirmation: nothing is written; the caller can change a slot
```

The actions the manager can take are defined in `config.Action`: asking for each slot, checking availability, asking for confirmation, creating, cancelling and rescheduling appointments, checking status, giving doctor and clinic information, offering alternatives, greeting, clarifying, escalating an emergency, and ending the conversation.

6.5 Error handling

- Backend failures come back as a structured result and are explained to the caller in plain words; technical errors are never read out.
- If the database is unreachable the assistant says so instead of pretending to book.
- Dead ends found in testing are closed: for example, answering "nahi" to "what is your appointment ID?" lists the caller's bookings instead of looping.

7. LLM Judge and Response Generator

7.1 Why an LLM is used

Template replies are correct but stiff, and do not adapt to the caller's wording. A local LLM rephrases the backend's result naturally. Running it locally through **Ollama** keeps patient data on the clinic's machine and costs nothing per call.

7.2 Model and settings

Setting	Value
Runtime	Ollama (tested with version 0.34.4), called over HTTP with <code>httpx</code>
Model	<code>llama3.2</code> (<code>OLLAMA_MODEL</code>)
Temperature	0.2 – it rewrites facts it is given, it does not invent
Maximum tokens	200 – a spoken reply is one or two sentences
Timeout	30 s, after which the template is used
Can be switched off	<code>OLLAMA_ENABLED=false</code> gives deterministic replies only

7.3 Judge, validation and decision process

```
backend result -> Ollama (prompt with the facts) -> response validator -> approved text
|
+-- rejected -> deterministic
template
```

The validator (`response_validator.py`) checks every time, date and appointment ID mentioned in the generated sentence against the backend result, rejects a failed operation described as a success, and rejects wrong relative dates, wrong weekdays, raw ISO dates, and a reply in the wrong language or script. If anything fails, the deterministic template (`fallback.py` , `dialog_templates.py`) is used.

7.4 Languages

Replies are produced in English, Roman Urdu or Urdu (`language.py`), following the caller's language unless `RESPONSE_LANGUAGE` fixes it. Templates exist for every action in all three languages, so the assistant answers in the caller's language even with the LLM switched off.

7.5 Integration with the Dialogue Manager

The Dialogue Manager decides *what* happens and the backend makes it happen; only then is the judge asked *how to say it*. Turns with no database operation (a slot question, a greeting) use templates by default, because there is no backend result to validate the model's wording against (`OLLAMA_TRANSLATE` turns that on for stronger models).

Measured behaviour. In the recorded live run against llama3.2 (reports/ollama_live.json), 21 replies were requested: 3 were accepted by the validator and 18 fell back to templates, with a median latency of 4.12 s. An earlier run recorded 9 of 21 accepted. The guard is doing its job – the small 3-billion-parameter model often mangles Roman Urdu or restates a time differently – and the templates carry most calls. A larger model would raise the acceptance rate.

8. Appointment Backend

The Appointment Backend (`appointment_backend/`) is the only code that creates or changes an appointment. The voice pipeline, the doctor dashboard and the administration area all go through its facade, `AppointmentBackend` .

Module	Responsibility
<code>appointment_service.py</code>	The facade used by every caller.
<code>booking_service.py</code>	The single path that creates appointments.
<code>cancellation_service.py</code>	Patient cancellation, clinic cancellation, and the doctor-leave cascade.
<code>reschedule_service.py</code>	Moving an appointment to a new date and time.
<code>availability_service.py</code>	Free slots: working hours minus leave minus existing bookings.
<code>validation.py</code>	Gates run before any write.
<code>statistics.py</code>	Counting appointments by status and period.
<code>result.py</code>	The uniform <code>OperationResult</code> .

8.1 Operations and endpoints

Operation	Public (voice) route	Dashboard / admin route
Create	<code>POST /appointments/book</code>	<code>POST /dashboard/appointments</code>
Retrieve	<code>GET /appointments/{id}</code> , <code>GET /patients/{id}/appointments</code> , <code>GET /doctors/{id}/appointments</code>	<code>GET /dashboard/appointments</code> , <code>GET /admin/appointments</code>
Check without writing	<code>POST /appointments/check</code>	—
Cancel	<code>POST /appointments/cancel</code>	<code>POST /dashboard/appointments/{id}/cancel</code> , <code>POST /admin/appointments/{id}/cancel</code>
Reschedule	<code>POST /appointments/reschedule</code>	<code>.../reschedule</code> on both
Complete	—	<code>.../complete</code> on both
Availability	<code>GET /doctors/{id}/availability?date=</code>	<code>GET /dashboard/availability</code> , <code>GET /admin/doctors/{id}/availability</code>
Doctor leave	<code>POST /doctors/unavailability</code>	<code>/dashboard/leave</code> , <code>/admin/doctors/{id}/leave</code> (+ <code>/preview</code>)

Appendix A lists all 72 routes.

8.2 Appointment statuses

Status	Meaning
<code>pending</code>	Created, not yet confirmed
<code>confirmed</code>	Booked (the assistant confirms before writing)
<code>rescheduled</code>	A live appointment that was moved
<code>completed</code>	Consultation finished (set from a dashboard)
<code>cancelled</code>	Cancelled by the patient or the clinic
<code>cancelled_by_doctor</code>	Cancelled because the doctor took leave that day

Records are never deleted: cancellation changes the status and keeps the history.

8.3 Business rules

- **Valid date and time:** no past dates; the time must be one of the doctor's slots for that weekday.
- **Doctor must be active:** a deactivated or removed doctor is refused and is not offered to callers.
- **Doctor must be working:** the weekly schedule (`schedules`) and leave dates (`doctor_unavailability`) are applied; the seeded clinic closes on Sunday.
- **No double booking:** availability is checked, then the write goes through a transactional `reserve_slot` that re-checks inside a Firestore transaction, so two callers racing for the same slot cannot both win.
- **Ownership:** a caller may only cancel or move their own appointment; a doctor may only act on appointments in their own diary.
- **Final states are final:** a completed or cancelled appointment cannot be changed again.

8.4 Doctor leave

Blocking a date runs one cascade: the date is written to `doctor_unavailability` (so new bookings are refused), every live appointment that day becomes `cancelled_by_doctor` , and one notification per affected patient is queued in `notifications` . The administrator sees a preview of exactly which appointments and patients will be affected before confirming. Re-opening the date accepts new bookings but does not resurrect the cancelled ones.

9. Firebase / Database

9.1 Technology

Google Cloud Firestore (Firebase), accessed only from the server through the **Firestore Admin SDK** (`firebase-admin` , `google-cloud-firestore`). A repository abstraction (`firebase/firebase_config.py`) has two implementations with the same interface: `FirestoreRepository` for real data, and `LocalRepository` , an in-memory store used for tests and offline demos (selected with `USE_LOCAL_DB` , or automatically when no credentials are configured, in which case demo data is seeded). Both stamp `created_at` and `updated_at` on every write.

9.2 Collections and fields

Collection	Key fields
<code>clinics</code>	<code>clinic_id</code> , name, address, city, phone, active – one record (<code>CLINIC_ID</code> , default C001)
<code>doctors</code>	<code>doctor_id</code> , name, specialization, qualification, experience_years, fee, phone, email, about, photo, aliases, <code>clinic_ids</code> , active, archived, notification_prefs
<code>schedules</code>	<code>schedule_id</code> , <code>doctor_id</code> , <code>clinic_id</code> , day, start_time, end_time, slot_duration, active
<code>doctor_unavailability</code>	id { <code>doctor_id</code> }_{date} , <code>doctor_id</code> , date, reason, active
<code>patients</code>	<code>patient_id</code> , name, phone
<code>appointments</code>	<code>appointment_id</code> , <code>patient_id</code> , <code>doctor_id</code> , <code>clinic_id</code> , date, time, status, cancelled_at, cancellation_reason, completed_at
<code>notifications</code>	<code>notification_id</code> , <code>patient_id</code> , <code>appointment_id</code> , <code>doctor_id</code> , type, channel, status, date, time, message, read_by_doctor
<code>conversation_sessions</code>	Dialogue state per session
<code>conversation_messages</code>	One document per message, phone numbers masked
<code>dashboard_accounts</code>	<code>account_id</code> , username, role, password_hash, password_salt, iterations

9.3 Relationships

```
clinics (one record)
├── doctors ── schedules (doctor_id)
│   ├── doctor_unavailability (doctor_id)
│   ├── dashboard_accounts (account_id = doctor_id)
│   └── appointments ── patients (doctor_id, patient_id)
│       └── notifications (appointment_id, patient_id, doctor_id)
```

This is a **single-clinic** system: every doctor belongs to the one clinic. Doctors keep a `clinic_ids` field containing that clinic for compatibility with the booking rules, but nothing in the system adds, selects or

switches clinics, and every screen and spoken reply reads the clinic's name through `ClinicService.primary()` .

9.4 Data flow

Writes happen only in the Appointment Backend and the dashboard APIs, through the services in `firebase/` . Reads for display go through the same services. The browser holds no Firebase credentials and never calls Firestore; it calls the FastAPI service, which calls Firestore.

9.5 Authentication to Firebase

The server authenticates with a service account, given either as three environment variables (`FIREBASE_PROJECT_ID` , `FIREBASE_CLIENT_EMAIL` , `FIREBASE_PRIVATE_KEY`) or as the path to the key file (`FIREBASE_CREDENTIALS_FILE`). `scripts/verify_firebase.py` checks the configuration and connectivity. No Firestore security-rules file is part of this repository; all access is from the server's Admin SDK.

10. Doctor Dashboard

A web application in `Dashboard/`, written in plain HTML, CSS and JavaScript with no build step, and served by the API at `/ui`. A doctor signs in with their username or doctor ID and sees **only their own data**; the server scopes every `/dashboard/*` request to the doctor in the session.

Page	What it does	API
Dashboard	Today's counters, today's list, who is due now, what is coming up, and their own appointment statistics by period.	<code>/dashboard/summary</code> , <code>/dashboard/statistics</code>
Appointments	Filter by scope, date and status; view, complete, reschedule, cancel; book a new appointment against real free slots.	<code>/dashboard/appointments</code>
Patients	The people in this doctor's diary, with visit counts and history; register a patient.	<code>/dashboard/patients</code>
Schedule	The weekly working hours callers are offered, and leave dates with the cancel-and-notify cascade.	<code>/dashboard/schedule</code> , <code>/dashboard/leave</code>
Doctor Profile	Specialization, qualifications, experience, fee, about. The clinic is shown as read-only information.	<code>/dashboard/profile</code>
Notifications	Messages queued for this doctor's patients, with their delivery state.	<code>/dashboard/notifications</code>
Settings	Account (photo, name, email, phone; username shown read-only), own password, alert preferences, appearance (light, dark, follow system), session.	<code>/dashboard/profile</code> , <code>/auth/password</code>

10.1 Statistics

The doctor sees Total, Confirmed, Pending, Completed and Cancelled counts (and Rescheduled, when there are any) for Today, This week, This month, All time or a custom range, a bar showing the split, and today's breakdown. The endpoint takes no doctor ID at all, so a doctor cannot request another doctor's numbers.

10.2 Keeping up to date

While open, the dashboard re-reads today's diary every 30 seconds and raises the alerts the doctor has switched on: a new booking by the assistant, a cancellation, or a changed time.

11. SuperAdmin Dashboard

The administrator signs in with the administrator ID and reaches a separate area (`/admin/*`). The two roles do not overlap: a doctor's token is refused by every administration route, and the administrator's token is refused by the doctor routes, because that data belongs to a doctor.

Single clinic. There is one clinic, many doctors, and their appointments. The administrator is the only person who can change the clinic's name, address, city and phone; the change reaches every doctor profile, every appointment detail and the assistant's spoken confirmation at once.

Area	Capabilities
Overview	Doctors on the register, bookable, deactivated and on leave today; what is in the diary now; clinic-wide statistics by period with a per-doctor table; today's appointments; recent activity.
Doctors	Search and filter the register; add a doctor (including username and password in the same step); edit profile, photo, fee and working hours; deactivate and activate; remove (archive, never delete) and restore. Clicking a doctor's name opens their appointments.
Doctor details	Profile; availability (working week and leave dates, with a preview of which appointments a new leave would cancel); appointments; and Sign-in , where the doctor's username and password are set.
Appointments	Every doctor's appointments, filtered by scope, doctor, date and search; cancel with a recorded reason, move to a free slot, or mark completed – each through the Appointment Backend.
Clinic	The one clinic's name, address, city and phone. A database left over from an earlier multi-clinic version can be consolidated onto the one record.
Settings	The administrator's own password, appearance and session.

11.1 What administrator actions do to the system

Action	Effect
Add a doctor	Writes the doctor record with generated name aliases, so the assistant recognises the spoken name at once; writes the working week; issues sign-in details if given. A name that collides with another doctor's is reported.
Deactivate	Sets <code>active: false</code> . New bookings are refused, the assistant stops offering the doctor, and sign-in is refused. Existing appointments are kept.
Remove	Archives the record (<code>archived: true</code>); appointments, patients and messages stay.
Set a password	Stores a hash; the doctor's other open sessions are signed out.
Cancel an appointment	Status becomes <code>cancelled</code> with the stated reason, the slot is freed, and a message for the patient is queued.
Give leave	Runs the leave cascade of Section 8.4.

11.2 Clinic and doctor statistics

`GET /admin/statistics` counts the clinic's appointments by status for a chosen period and returns one row per doctor (total, confirmed, pending, completed, cancelled). Choosing a doctor narrows the headline figures to them while the table still lists everybody for comparison. The doctor rows always add up to the clinic total; this is tested.

12. Speech-to-Text (STT)

Status: not implemented. There is no speech-recognition component in this repository. No module converts audio to text, and no test exercises one. The recorded performance measurements list speech-to-text as "not measured: no STT integration exists in the codebase".

What exists instead. The pipeline accepts the patient's words as text at `POST /dialog/message`, `POST /voice/message` and in the interactive `voice_pipeline.py`. Everything downstream – intent detection, dialogue, backend, database, reply – works on that text, so an STT stage can be added in front without changing it.

Language support that would apply. The text stages already handle English, Urdu in Arabic script, Roman Urdu and mixed input (Section 5.2), so an STT model's output in any of those forms can be consumed directly.

Considerations for adding it (future work): telephone audio is 8 kHz narrow-band (G.711 μ -law or A-law); recognition latency adds directly to each turn; and an Urdu-capable model such as Whisper would be needed.

13. Text-to-Speech (TTS)

Status: not implemented. There is no speech-synthesis component in this repository.

The reply is produced as **text** in English, Roman Urdu or Urdu (Section 7.4), returned by `/dialog/message` and `/voice/message`. Dates and times are already written in spoken form (for example "Thursday, 1 October at 6:00 PM", never an ISO date), so a TTS engine can read the text as it is. Adding TTS is future work.

14. Asterisk / SIP Telephony

Status: not implemented. There is no Asterisk configuration, dial plan, SIP account, AudioSocket or AGI integration in this repository.

The integration point that exists. `POST /voice/message` is designed as the endpoint a telephony layer would call on each turn: it takes a session ID, the transcribed text and an optional patient ID, runs the whole pipeline, and returns the sentence to speak together with the intent, the backend result and the judge's decision. A future telephony bridge would connect Asterisk (receiving the call), an STT stage, this endpoint and a TTS stage.

The caller would be identified by phone number (`PatientService.find_by_phone` exists and is used today); in production the caller ID from the telephony layer would supply it.

15. Testing

The test suite uses the standard-library `unittest` framework and runs with one command, `python scripts/run_tests.py`, which groups the tests by component and writes machine-readable results to `reports/`. Tests run against the in-memory repository, so they never touch the real database. Tests that need live Firestore or a running Ollama are opt-in (`--live`) and are counted as skipped otherwise.

15.1 Offline test results

Run on 25 September 2026, on the repository as submitted:

Category	Test modules	Tests	Passed	Failed	Skipped
Intent detection	test_intents, test_roman_urdu, test_intent_audit	45	45	0	0
Dialogue	test_dialog, test_entities, test_naturalness, test_scenarios, test_emergency_safety	112	112	0	0
Appointment backend	test_backend, test_appointments	61	60	0	1
Firebase	test_firebase, test_firestore_live	40	31	0	9
Ollama / wording	test_ollama, test_ollama_live	54	51	0	3
End-to-end	test_integration, test_api	25	25	0	0
Dashboards	test_dashboard_api, test_admin_api, test_statistics	122	121	0	1
Security	test_security, test_data_safety	11	11	0	0
Total		470	456	0	14

The 14 skipped tests are the live Firestore and Ollama tests, which need `--live` and real services.

15.2 What the tests cover

- **Unit:** entity extraction (dates, times, doctor names), slot logic, statistics arithmetic (every record counted exactly once, month-end and leap-year boundaries), password hashing rules.
- **Intent detection:** the trained model on fixed phrasings in each language, Roman-Urdu spelling variants, a hand-written audit set, and a check that no challenge phrase leaked into training data.
- **Appointments:** booking, double-booking refusal, cancellation keeping the record, rescheduling rules, completed appointments being final, and the leave cascade with its notifications.
- **Integration and end-to-end:** whole conversations through the real mBERT model and Dialogue Manager to a booking, and the HTTP API.
- **Dashboards:** sign-in for both roles; a doctor never seeing another doctor's data (403); the administrator adding a doctor who is then found by name and bookable through the voice route; deactivation refusing bookings; removal archiving instead of deleting; credentials; the single clinic

and its name reaching every reader; a five-appointment leave scenario (previewed, cancelled, five patients notified, history kept); and statistics matching the records by hand.

- **Security:** no key or password in any source or report file, `.env` and the service-account file ignored by Git, credentials never written to logs.

15.3 Browser tests

Two scripts in `tests/browser/` drive the real dashboard in headless Chrome over the DevTools protocol and fail on any JavaScript error:

Script	Checks	Result	Covers
<code>dashboard_checks.js</code>	59	59 passed	Sign-in screen, both roles, the doctor register, adding, viewing, deactivating, removing and restoring a doctor, settings, role separation, every doctor page, phone-width layout
<code>admin_checks.js</code>	51	51 passed	The clinic record, clinic-wide appointments with cancellation, the leave preview, statistics, credentials, a doctor changing their own password, what a doctor may not do, the administrator's password

15.4 Verification against real data

- **Statistics.** Read-only against the live Firestore project: 4 appointments and 5 doctors; the counts reported by the statistics module matched a hand count exactly (total 4, confirmed 1, completed 1, cancelled 2), the doctor rows summed to the clinic total, and each period's count matched a filter of the records.
- **Voice pipeline.** A doctor added through the administration area was booked in a Roman-Urdu conversation ("dr uzma se appointment chahiye" → "kal" → "shaam 5 baje" → "haan"), and the appointment appeared in the administrator's view and in that doctor's own dashboard.
- **Firestore round trip.** An appointment booked through the public route against live Firestore appeared in the doctor dashboard, was cancelled there, kept its record as `cancelled`, and freed its slot.

16. Security

16.1 Secrets and environment variables

- All secrets come from environment variables or a `.env` file that is ignored by Git. `.env.example` lists the variable names with empty values.
- The Firebase service-account file is ignored by Git by name pattern, and every other JSON file is ignored unless explicitly listed.
- The browser holds no Firebase credential, API key or server address; the dashboard never displays configuration or connection details.
- A test (`tests/test_security.py`) scans the source, the reports and this PDF for private keys and service-account e-mail addresses, and checks that the secret files are ignored.

16.2 Authentication

- `POST /auth/login` accepts a username, a doctor ID or the administrator ID, and returns an opaque random token (`secrets.token_urlsafe(32)`) sent as a bearer token.
- Passwords are stored in `dashboard_accounts` as PBKDF2-HMAC-SHA256 hashes (120,000 iterations, a 16-byte random salt per account) and compared in constant time. No password or hash is ever returned by the API.
- `DASHBOARD_PASSWORD` and `SUPERADMIN_PASSWORD` are only *starting* passwords: they open an account that has never had one set, and stop working as soon as one is.
- Wrong ID and wrong password give the same answer, so the sign-in form cannot be used to discover valid IDs. A deactivated doctor cannot sign in.

16.3 Authorisation

- Two roles, enforced on the server: `doctor` for `/dashboard/*` and `superadmin` for `/admin/*`. Each is refused (403) by the other's routes.
- A doctor's routes take the doctor ID from the session, never from the request; another doctor's appointment returns 403.
- A doctor cannot change their username, the clinic's details or anyone else's credentials; there is no route for the first and the API refuses the others.
- Changing a password signs the account out of its other sessions.

16.4 API and Firebase security

- CORS is limited to the origins in `DASHBOARD_ORIGINS` , without credentials.
- Firestore is reached only by the server's Admin SDK.

Known limits. Sessions live in the API process, so restarting it signs everyone out; there is no sign-in rate limiting and no self-service password reset. The public voice-pipeline routes (/appointments/* , /dialog/* , /voice/message) are not authenticated, because they are meant to sit behind the telephony layer on a private network. Production would add Firebase Authentication and network restrictions.

17. Technologies Used

Category	Technology	Version tested
Programming languages	Python; JavaScript (browser, and Node.js for the browser tests); HTML and CSS	Python 3.12.10; Node.js 24.16
NLP model	mBERT (bert-base-multilingual-cased) fine-tuned for 31 intents	—
Deep learning	PyTorch, Hugging Face Transformers	torch 2.14.0 (CUDA 12.6), transformers 5.15.0
Data and metrics	pandas, NumPy, scikit-learn, Matplotlib	3.0.5, 2.5.1, 1.9.0, 3.11.1
LLM	Ollama running llama3.2	Ollama 0.34.4
STT	Not implemented	—
TTS	Not implemented	—
Backend	FastAPI, Uvicorn, Pydantic, httpx, python-dotenv	0.141.1, 0.52.3, 2.13.4, 0.28.1, 1.2.2
Database	Google Cloud Firestore via the Firebase Admin SDK	firebase-admin 7.5.0, google-cloud-firestore 2.30.0
Telephony	Not implemented (Asterisk/SIP planned)	—
Frontend	HTML, CSS and JavaScript without a framework or build step; Inter web font	—
Testing	unittest ; headless Chrome for browser tests; pypdf for the report check	pypdf 6.16.0
Version control	Git and GitHub, GitHub CLI	git 2.54, gh 2.95
Hardware used	NVIDIA GeForce GTX 1660 SUPER (6 GB) for training	—
Tools	Visual Studio Code, PowerShell, Google Chrome	—

18. Performance Measurements

Measured by `scripts/measure_performance.py` on 14 September 2026 on the development machine (NVIDIA GTX 1660 SUPER), recorded in `reports/performance.json` :

Stage	Calls	Median	95th percentile
mBERT intent detection (GPU)	60	9.97 ms	13.54 ms
first call, including loading the model	1	7.58 s	
Dialogue Manager logic (in memory)	60	0.30 ms	0.45 ms
Whole turn without the LLM (in memory)	30	9.66 ms	12.57 ms
Backend booking (in memory)	8	0.07 ms	0.07 ms
Firestore: read a doctor	5	2.34 s	2.39 s
Firestore: availability for a date	5	5.85 s	5.86 s
Ollama judge (Roman Urdu)	3	3.13 s	3.13 s

Live Firestore operations measured in the live test run (`reports/firestore_live.json`): booking median 6.13 s (8 calls), rescheduling 4.41 s, doctor leave 5.04 s, cancellation 2.23 s.

Interpretation. The language model and the classifier are fast; the network round trips to Firestore dominate each turn, because every validation gate is a separate read. Reading a doctor's schedule once per call, or caching the doctor and clinic records, is the most effective improvement available.

19. Limitations and Future Work

Area	Current state	Future work
Telephony and voice	STT, TTS and Asterisk/SIP are not implemented; the pipeline is text in, text out.	Add an Asterisk bridge, an Urdu-capable STT model and a TTS engine in front of and behind <code>/voice/message</code> .
Notifications	Patient messages are queued in Firestore and shown on the dashboards; nothing is sent.	Implement the <code>NotificationSender</code> interface against an SMS or voice provider, with credentials from the environment.
Intent model	Macro-F1 0.836; short information questions and conversational closers are weakest; emergency recall 0.55, backed by the phrase safety net.	More natural phrasings per intent (30–50), a fresh held-out set, and more emergency examples with the booking phrase first.
LLM wording	<code>llama3.2</code> wording is accepted for a minority of replies; templates carry most calls.	A larger local model (for example 7–8B parameters).
Authentication	Prototype sessions in the API process; no rate limiting or self-service reset.	Firebase Authentication, rate limiting, and authenticated voice routes.
Performance	Several seconds per Firestore-backed turn.	Fewer round trips and caching of doctor and clinic records.
Data model	Patients hold a name and phone only; appointments have no reason or notes field.	Extend the records when the clinic needs those fields.

20. Conclusion

The implemented system understands appointment requests in English, Urdu, Roman Urdu and mixed speech with a fine-tuned mBERT classifier (test accuracy 0.898, macro-F1 0.836 over 31 intents), holds a stateful conversation that asks only for what is missing and confirms before acting, applies the clinic's scheduling rules in a single tested backend, and stores everything in Firebase Firestore. A local language model phrases replies naturally but is never allowed to change a fact.

Around that pipeline, doctors have a dashboard for their own day, and a single administrator manages the one clinic, its doctors, their credentials, their leave, every appointment and clinic-wide statistics, with the separation between the two roles enforced on the server. The system is covered by 470 automated tests, of which none fail, and 110 browser checks.

The telephone layer – Asterisk/SIP, speech recognition and speech synthesis – remains future work. The pipeline was built so that these stages attach at its two ends without changes to the parts described in this report.

Appendix A – HTTP API Reference

All 72 routes served by the FastAPI application. Interactive documentation is at `/docs` when the service is running.

Method	Path	Purpose
Service and conversation		
GET	<code>/</code> , <code>/health</code>	Service status; backend and model health
POST	<code>/dialog/message</code>	One conversation turn: text in, reply and state out
POST	<code>/dialog/reset</code> , <code>/dialog/end</code>	Reset or end a session
GET	<code>/dialog/transcript/{session_id}</code>	Stored transcript, phone numbers masked
POST	<code>/voice/message</code>	Full pipeline for a telephony layer (text in, reply text out)
POST	<code>/intent/predict</code>	The mBERT classifier on its own
Appointment Backend (voice pipeline)		
POST	<code>/appointments/book</code> , <code>/cancel</code> , <code>/reschedule</code> , <code>/check</code>	Book, cancel, move, or check without writing
GET	<code>/appointments/{appointment_id}</code>	One appointment
GET	<code>/patients/{patient_id}/appointments</code>	A patient's appointments
GET	<code>/doctors</code> , <code>/doctors/{doctor_id}</code>	Active doctors; one doctor
GET	<code>/doctors/{doctor_id}/availability</code> , <code>/appointments</code>	Free slots on a date; a doctor's appointments
POST	<code>/doctors/unavailability</code>	Block a date (leave cascade)
Authentication		
POST	<code>/auth/login</code> , <code>/auth/logout</code>	Start and end a session
GET	<code>/auth/session</code>	Who is signed in, and their role
POST	<code>/auth/password</code>	Change your own password
Doctor dashboard (doctor role, own data only)		
GET	<code>/dashboard/summary</code> , <code>/dashboard/statistics</code>	Today at a glance; counts by status and period
GET, POST	<code>/dashboard/appointments</code>	List; book
GET	<code>/dashboard/appointments/{id}</code>	One appointment

POST	/dashboard/appointments/{id}/cancel , /reschedule , /complete	Act on one
GET	/dashboard/availability	Free slots
GET, POST	/dashboard/patients ; GET /dashboard/patients/{id}	Patients
GET, PATCH	/dashboard/profile , /dashboard/schedule	Profile; weekly hours
GET, POST	/dashboard/leave ; DELETE /dashboard/leave/{date}	Leave dates
GET	/dashboard/notifications	Queued patient messages
POST	/dashboard/notifications/{id}/read , /read-all	Mark read
Administration (superadmin role)		
GET	/admin/summary , /admin/statistics	Overview; clinic and per-doctor counts
GET, POST	/admin/doctors	Register; add a doctor
GET, PATCH, DELETE	/admin/doctors/{id}	Details; edit; remove (archive)
POST	/admin/doctors/{id}/status , /restore	Activate or deactivate; restore
GET, PATCH	/admin/doctors/{id}/credentials , /schedule	Sign-in details; weekly hours
GET	/admin/doctors/{id}/appointments , /availability	A doctor's diary; free slots
GET, POST	/admin/doctors/{id}/leave ; GET /leave/preview ; DELETE /leave/{date}	Leave, with preview
GET, PATCH	/admin/clinic ; POST /admin/clinic/consolidate	The one clinic
GET	/admin/appointments , /admin/appointments/{id}	Every appointment
POST	/admin/appointments/{id}/cancel , /reschedule , /complete	Act on one

Appendix B – Configuration Variables

Names only. Values are set in the server's environment or in a `.env` file that is never committed; `.env.example` is the template.

Variable	Purpose
<code>FIREBASE_PROJECT_ID</code> , <code>FIREBASE_CLIENT_EMAIL</code> , <code>FIREBASE_PRIVATE_KEY</code>	Service-account credentials (option A)
<code>FIREBASE_CREDENTIALS_FILE</code>	Path to the service-account key file (option B)
<code>USE_LOCAL_DB</code>	Force the in-memory database
<code>CONFIDENCE_THRESHOLD</code> , <code>HIGH_CONFIDENCE</code> , <code>INTENT_SWITCH_THRESHOLD</code>	Intent confidence bands (0.60, 0.80, 0.75)
<code>EMERGENCY_SAFETY_NET</code>	Phrase-based emergency escalation (on by default)
<code>SESSION_TIMEOUT_MINUTES</code>	Dialogue session lifetime (15)
<code>OLLAMA_BASE_URL</code> , <code>OLLAMA_MODEL</code> , <code>OLLAMA_TIMEOUT</code> , <code>OLLAMA_TEMPERATURE</code> , <code>OLLAMA_MAX_TOKENS</code> , <code>OLLAMA_ENABLED</code> , <code>OLLAMA_TRANSLATE</code>	The local LLM
<code>RESPONSE_LANGUAGE</code>	Reply language, or <code>auto</code> to follow the caller
<code>DASHBOARD_PASSWORD</code> , <code>DASHBOARD_PASSWORD_<DOCTORID></code>	Starting passwords for doctor accounts
<code>SUPERADMIN_ID</code> , <code>SUPERADMIN_PASSWORD</code>	The administrator's ID and starting password
<code>DASHBOARD_SESSION_HOURS</code> , <code>DASHBOARD_ORIGINS</code>	Dashboard session lifetime; allowed browser origins
<code>CLINIC_ID</code>	The document that holds the one clinic (C001)
<code>NOTIFICATION_PROVIDER</code>	Reserved for an SMS or call provider (not implemented)
<code>APPOINTMENT_ID_PREFIX</code> , <code>TIMEZONE</code> , <code>LOG_LEVEL</code> , <code>INTENT_MODULE_DIR</code>	Appointment IDs, time zone (Asia/Karachi), logging, model location